Small Fine-Tuned Planners with Execution-Verified Data and Calibrated Abstention for Tabular Canonicalization

Ricardo Alanis
ricardo.alanis@gmail.com

June 2026

Abstract

Cleaning messy tabular data—particularly *canonicalization*, the merging of inconsistent surface forms such as `USA`/`U.S.A`/`united states` into one canonical value—resists rule-based automation and is routinely done by hand. We present ScrubData, a system in which a small ($\leq$4B-parameter) language model acts as a *planner*: it reads an aggregated column profile (per-value frequency counts, invariant to row count) and emits a structured JSON cleaning plan that a deterministic executor applies, making every change auditable and reversible. Three design decisions carry the result. (1) *Execution-verified training data*: every synthetic training example is kept only if executing its plan provably recovers the known-clean table. (2) *Reference grounding*: canonical forms are never free-generated; values are reconciled against type-scoped reference taxonomies (GeoNames, ISO) with fuzzy retrieval. (3) *Calibrated abstention*: below a confidence threshold the system declines to act and flags values for review; we show the confidence is selective-prediction-grade (AURC 0.120, precision rises monotonically with threshold). Extending abstention to the plan itself — a deterministic verifier that gates every proposed mapping — yields a zero-configuration, zero-label system that repairs 41% of a real benchmark's errors at 0.905 precision, with every declined merge surfaced for review.

Beyond the verifier, four deterministic capabilities — profile-level *suspect surfacing* for high-cardinality columns, reconciliation against a pluggable *entity reference* built from open vocabularies, *cross-row majority voting* over repeated-entity groups, and *convention-conservatism* gates that refuse to re-format internally consistent columns — carry the system to never-seen tables: across a 42-pair benchmark the unseen-source macro F1 is 0.363 at 0.0219 damage, with grouped-entity tables recovered at 0.955–0.957 F1 and *zero* damage, and *zero silent edits* across 35 wild tables plus a 239-table trust audit. Our second finding is negative and load-bearing: five supervised fine-tuning iterations and a three-arm GRPO pilot (with a random-reward control) all failed to make the *model weights* contribute to held-out generalization — the deterministic machinery, gated by the verifier, is what generalizes, a result independently corroborated by three concurrent studies. The same architecture covers PII protection: a 44M token classifier types columns from bare cell values (100% on names/addresses), and deterministic masking yields zero residual detectable PII. All components run locally on commodity hardware; no data leaves the machine.

1 Introduction

A large share of practical data work is cleaning: a sales export where the same country is spelled four ways, a hospital roster where `birminghxm` should be `birmingham`, a CRM dump with mixed date formats and duplicated contacts. The fuzzy half of this work—recognizing that distinct surface forms denote the same entity—is exactly what rules do poorly and humans do slowly.

Large language models can do this fuzzy matching, but deploying them as cell editors has three problems. First, *trust*: a model that edits cells directly can silently corrupt data, and its errors are unauditable. Second, *cost and privacy*: shipping every row of a private table to a hosted frontier model is expensive and often unacceptable. Third, *hallucination*: asked for a canonical form, a generative model will invent one, and on tail entities it will invent wrong ones.

ScrubData addresses all three with an architecture in which the model never touches data. A profiler aggregates each column into a value-frequency distribution; a small local model reads the profile and *proposes* a JSON cleaning plan; a deterministic pandas executor *applies* it. The plan is the complete, inspectable, reversible specification of every change—there are no silent edits by construction (§3). Because the prompt scales with the number of *distinct* values rather than rows, a million-row table profiles like a hundred-row one.

This paper makes six contributions:

1. **A planner/executor decomposition for data cleaning** with an auditable trust contract: the model proposes, a deterministic engine executes, every decision is logged with lineage (§3).

2. **Execution-verified synthetic supervision**: a data-generation method in which every training example is validated by running the executor on the (dirty table, plan) pair and checking that the known-clean table is recovered; non-recovering examples are discarded (§3.2).

3. **Reference-grounded canonicalization with calibrated abstention**: canonical forms come from type-scoped reference taxonomies via fuzzy retrieval with an ambiguity margin; below-threshold matches abstain and surface for review. We characterize the confidence with risk–coverage analysis (§3.3, §5.8).

4. **An un-gameable evaluation**: a 65-dataset suite (real-error benchmarks plus seeded error injection over 20 harvested open-data domains) scored with a churn-neutral, convention-tolerant metric that cannot be inflated by mass rewriting; we report two metric artifacts that our own ablations exposed and corrected (§4).

5. **Four deterministic capabilities that carry never-seen-table generalization**: bounded suspect surfacing for high-cardinality columns, generic entity-reference reconciliation with an exact-hit typing floor, cross-row majority voting with a false-consensus guard, and convention-conservatism gates — each motivated by a measured failure regime and gated by the verifier (§3.5, §5.4). Released with them is WildClean, to our knowledge the first cleaning benchmark that scores damage and silent edits alongside repair F1.

6. **A negative result we argue is the load-bearing one**: five further supervised fine-tunes and a three-arm GRPO pilot with the executor as a verifiable reward all failed to move held-out generalization — in this protocol class, the model weights contribute format competence and little else; the deterministic machinery and plan-level verification carry the capability (§5.5).

We deliberately report a negative-flavored finding alongside the positive ones: on *injected* typos, classical frequency clustering remains a strong baseline—by construction, injection places the canonical form in the column, which is clustering's ideal regime. The advantage of grounding is concentrated where it matters: real errors, tail entities absent from the column, and adversarial near-misses where acting at all is wrong (§5).

2 Related Work

Error detection and repair. Raha and Baran [1] established configuration-free error detection and correction benchmarks (hospital, beers, flights, rayyan), which we adopt as out-of-distribution evaluation. HoloClean [2] combines integrity constraints, external reference data, and statistics in probabilistic repair, demonstrating that external signals can veto statistically plausible but wrong repairs—an insight our reference-veto inherits. GARF [3] learns repair rules self-supervised from the data itself; it also demonstrates the structural limit we observe for frequency-only methods: a lone categorical column offers no co-occurring signal to vote against an error.

The 2025–26 landscape. Post-Cocoon work concentrates on zero-label *detection*: ZeroED [7] (cloud-LLM cluster labeling, hospital detection F1 0.81, collapsing to 0.27 on smaller models), ForestED [8] (LLM-induced decision trees, 0.756), and Auto-Test [9] (corpus-mined semantic-domain constraints, no LLM at inference) — none performs zero-label *repair*. GIDCL [10] sets the labeled-class repair ceiling (hospital 0.97 with 20 labels and a LoRA trained per cleaned table); Cocoon [6] remains an unreproduced preprint (15 citing papers, none a reproduction). Three concurrent results independently corroborate this paper's central negative finding that machinery, not weights, carries cleaning generalization: an agentic-cleaning ablation where a deterministic harness contributes most of the gain at 4B scale [11], a study showing even frontier models cannot correct table distortions without explicit priors [12], and a large multi-agent-debate evaluation in which LLM self-critique *degrades* repair and only an adversarially separate, execution-grounded critic helps [13] — the architecture our verifier instantiates.

LLMs for data wrangling. Narayan et al. [4] showed frontier foundation models handle entity matching and imputation few-shot; Jellyfish [5] and Table-GPT [16] fine-tune mid-size models for data tasks. RetClean [17] is closest in spirit: retrieval from data lakes grounds cell repair, with the key empirical split that parametric knowledge suffices on world-known head values but collapses on the tail—motivating retrieval. Our work differs in the planner/executor decomposition (the model emits no cell values, only plans), in execution-verified supervision, and in the calibrated-abstention contract.

Entity linking over tables. TURL [18] and TableLlama [19] inject candidate entities into table understanding; Belotti et al. [20] show retriever coverage is the accuracy ceiling for table entity disambiguation and that long candidate lists hurt smaller models. RACOON [21] shows inference-time KG retrieval lifts a frozen model substantially, supporting our choice to ground at inference rather than bake aliases into weights (TURL's out-of-domain collapse is the cautionary result). MTab [22] established type-constrained matching with abstention in semantic table annotation.

Clustering-based cleaning tools. OpenRefine's key-collision (fingerprint) and nearest-neighbour clustering are the de-facto practitioner baseline; we reimplement both faithfully (including blocking) and compare head-to-head.

Selective prediction. Risk–coverage analysis and calibration metrics [23] formalize "knowing when not to act"; to our knowledge their application to data-cleaning merge decisions is new.

Small specialized models. OpenMed [24] fine-tunes sub-500M encoders to state-of-the-art biomedical NER, the sister result to our thesis that small specialized models beat large generic ones on narrow structured tasks; we adopt their released PII token classifiers for column typing (§3.6).

3 Method

3.1 Planner / executor decomposition

A *profiler* reduces each column to a typed summary: detected semantic type, missing counts, issue flags, and a value–frequency distribution capped at 80 distinct values (high-cardinality columns are summarized by their head). The *planner*—either a deterministic heuristic or our fine-tuned 4B model—maps the profile (plus three sample rows) to a JSON plan: a list of per-column operations drawn from a closed vocabulary (`canonicalize_categories` with an explicit mapping, `parse_date`, `standardize_phone`, `mask_pii`, …), table operations, and review flags. The *executor* applies the plan with pure pandas transforms. The plan is the only channel through which data changes: every diff is attributable to a named operation with a rationale, the original table is never mutated, and abstentions are first-class plan objects. We export per-run decision summaries as OpenTelemetry GenAI spans.

3.2 Execution-verified synthetic supervision

Training pairs are generated by corrupting clean synthetic tables with realistic noise (casing, aliases, single-character typos with Zipf-distributed long-tail categorical columns of 30–80 distinct values) while recording the ground-truth plan. The defining step is *verification by execution*: a candidate example is kept only if $\textsc{Execute}(\text{dirty}, \text{plan}) = \text{clean}$ cell-for-cell. This closes the loop between supervision and semantics—a plan that would not actually clean the table can never become a training label. We augment with real supervision derived from paired dirty/clean benchmarks by aligning cells and keeping only *learnable* canonicalizations (a surface form that is a string variant of its target and never a legitimate value elsewhere), which excludes unlearnable per-cell corrections such as divergent flight times. The fine-tune is QLoRA (rank 32) over Qwen3-4B-Instruct in bf16; one practical finding is that the base model's tool-calling prior dominates free-running generation even after convergent fine-tuning (loss 0.16) and must be suppressed at decode time by banning the two tool-call tokens.

3.3 Reference-grounded canonicalization with abstention

For columns whose values reconcile to a known concept type (countries, administrative regions, cities), canonical forms are never generated: a fuzzy retriever (normalized edit similarity with first-character blocking and length prefilters) matches each distinct value against the type-scoped reference (ISO/pycountry; GeoNames cities500, 196k entries). A value maps to a canonical only if (i) similarity clears a threshold τ=0.84, (ii) the best–second-best margin clears 0.03 (ambiguity veto: a value equally close to `Box` and `Boaz` abstains), and (iii) the canonical is cast to the column's observed case convention. Near-misses ($0.70 \le s < \tau$) are surfaced as review flags. The same wrapper grounds the *model* planner: for reference-typed columns the model's free-generated mapping is replaced by the grounded one, so the model can add coverage but never invent a canonical for a grounded type.

3.4 Plan-level selective prediction: the verified union planner

Grounding constrains reference-typed columns, but the planner's *free* canonicalization mappings on non-grounded columns remain unguarded—and they are where real-data precision dies (the fine-tune's raw hospital plan: 0.185 precision at 0.475 recall). Rather than retrain, we extend abstention to the plan itself. A deterministic *verifier* scores every proposed mapping entry $raw \to canon$ with

contract-preserving evidence (no cell values emitted, no gold access): three hard gates distilled from the model's measured failure classes—a value occurring $\geq$3 times is data, not a typo (*errors are rare*); the target must be a frequent column value clearly dominating the source (no mapping one typo onto another); digit-bearing codes repair only when the letter part is near-identical—then a confidence combining edit similarity with frequency support. Entries below a threshold τ are dropped to review flags; abstention stays first-class. Sweeping τ yields a plan-level precision–coverage curve. The shipped composition, the *verified union planner*, is the verifier-gated model plan (τ=0.5) unioned with the grounded heuristic's mappings (the model wins per surface form); the same code path is the product default.

3.5 Visibility and consensus: four deterministic capabilities

Four further mechanisms, each motivated by a measured failure regime on never-seen tables, complete the deterministic machinery. **(a) Suspect surfacing.** The profile's value-frequency view is capped, so high-cardinality columns hide their dirty cells from any planner. Every column profile now carries a bounded `suspect_values` section: rare anomalous surfaces with evidence-backed repair candidates (frequency dominance, edit similarity, reference membership). The heuristic planner repairs from suspects under a strict verifier bar (τ_{hc}=0.8) and flags the rest. **(b) Generic entity reference.** Open vocabularies (SemTab ToughTables aliases — derived excluding our benchmark tables; MusicBrainz search-hint misspellings; RxNorm; Wikidata; ROR) register as a pluggable reference type. Because the reference is broad, entity-typing a column additionally requires that $\geq$20% of its distinct values match the reference *exactly* — fuzzy coverage alone overfires on name-like columns (measured). This resolves the regime where every surface in a column is unique (no in-column frequency signal exists at all): five such benchmark tables go from 0.0 to 0.955–0.957 F1 at *zero* damage. **(c) Cross-row majority voting.** Tables that repeat a real-world entity across rows (a flight reported by many sources) carry their own repair signal. A detection step finds compact-token key columns with small groups (median multiplicity 3–30) and columns whose groups show *majority-bearing* disagreement with per-group information; a table-level operation then resolves thin dissenting minorities to the group majority. A *false-consensus* guard declines when minority shares look like legitimate correlated updates rather than reporting errors (mean minority share $\geq$0.25) — a flat volume cap was measured to destroy the legitimate dense-disagreement regime and replaced. **(d) Convention conservatism.** The planner never re-formats an internally consistent column: date and percent ops are gated on dominant-shape inconsistency (digit and alpha runs collapsed; 90% rule), ZIP/postal-named columns are never typed as phones or dates, and Excel-serial date typing requires a date-suggestive column name. Suppressed minority values surface as review flags — abstention is visible, never silent. The verifier enforces the same gates on model-emitted plans at the verification boundary.

3.6 PII as a second task instance

The identical contract covers PII: a deterministic tier types columns by checksum and pattern validators (Luhn, IBAN mod-97, SSN/email/phone) over distinct values; an optional 44M OpenMed-PII token classifier extends coverage to names and addresses, gated by a sensitive-type allowlist and a column-level coverage vote. High-sensitivity checksum-confirmed columns are masked format-preservingly (keep-last-four), contact columns are flagged for review, and identifier columns are excluded from numeric reformatting. Masking, salted hashing, and join-stable pseudonymization are deterministic executor operations.

4 Evaluation Design

Suite. Five real-error benchmarks (Raha) plus seeded error injection (typo/OCR/case/whitespace) over 15 harvested open-data domains (NYC, Chicago, SF, LA, Seattle, Texas, WA portals; GitHub) ≈ 65 datasets per seed. We aggregate as a *double macro*—mean over error types of mean over datasets, harmonically combined with the domain macro—so no single table or error type dominates.

Churn-neutral metric. A cell change that is case/whitespace-equivalent to the input but does not restore the gold counts as nothing: not a fix, not a change, not damage. Without this, mass case-rewriting inflates precision (we observed +0.12 NORTH from *removing* case matching before the correction); with it, fixing a case-injected error requires actually acting. We additionally report *damage*—the rate of semantically corrupting clean cells—and an adversarial *abstain slice* whose traps are garbage strings (not single-edit variants of any reference entity; an earlier trap set mis-scored grounding for correctly mapping `Boazz`→`Boaz`). We report both repairs of these metric artifacts as evidence that gameability must be tested, not assumed.

Real vs. injected. Injected typos are in-distribution for frequency clustering by construction (the canonical is present and dominant in the column), so we report the real-error and injected slices separately.

5 Results

5.1 Small fine-tuned planner vs. large generic model

On frozen synthetic gold, the fine-tuned 4B planner reaches canonicalization micro-F1 0.803 ± 0.009 (95% CI, 3 training seeds) — versus 0.452 for a much larger zero-shot generalist prompted identically and 0.152 for the rule heuristic (best single run 0.901; operation-F1 0.957, JSON validity 0.950). On real hospital typos the synthetic-only fine-tune scores 0.000 repair recall; adding 20% real-derived supervision lifts it to 0.424, and a data-scaling iteration (tripling the real-derived share from three paired benchmarks) reaches 0.475 recall at 0.185 precision—approaching the 0.51 of a frontier-scale zero-shot model. The scaling gain is seed-robust: +0.09 canonicalization F1 over the base mix under identical protocol, with non-overlapping 3-seed confidence intervals. Real, execution-verified pairs are what transfer: the same iteration found frequency-derived and algorithm-cleaned labels both *reduce* quality, consistent with our grounding thesis.

5.2 Grounding vs. clustering

With the errors-are-rare frequency gates now in both paths, grounding and frequency clustering are comparable on hospital alone (repairs-only, churn-neutral: 0.845 precision at 0.257 recall grounded vs 0.871 at 0.293 clustering—hospital's dominant errors are in-column typos, clustering's best case). Grounding's margin appears where references matter: across the five-benchmark real-error macro it reaches 0.174 versus 0.135 for the frequency-clustering ablation (+29%), and it carries the behavioral guarantees below. On the full suite against OpenRefine (Table 1), the result splits cleanly by regime, and we report both. On the *real-error* slice—the regime the tool exists for—grounded cleaning reaches REAL-F1 0.174, 3.0× OpenRefine kNN (0.058) and 4.5× fingerprint (0.039), with seed CIs of ± 0.003. On the *injected* slice, fingerprint clustering wins (0.282 vs 0.214) at near-zero damage: our case/whitespace injectors are exactly the perturbations key-collision normalizes away, so this is its home game and we say so. kNN clustering—the method that, like us, attempts typo repair—loses on both slices while incurring the highest damage among baselines (0.096), the

Table 1: Wide-suite comparison, 3 injection seeds, churn-neutral metric. NORTH is the double-macro harmonic mean; REAL-F1 is the real-error slice. (Filled from the final run.)

System	NORTH	±95%CI	REAL-F1	INJ-F1	damage	abstain
Grounded (ours)	0.203	0.003	**0.174**	0.214	0.104	1.000
OpenRefine fingerprint	0.211	0.000	0.039	0.282	0.001	1.000
OpenRefine kNN	0.122	0.002	0.058	0.148	0.096	1.000
Verified union 4B (shipped)†	–	–	0.142	–	0.015	1.000
Baran (oracle det. + 20 labels)‡	–	–	0.811	–	0.003	–
Jellyfish-13B (ED+DI)‡	–	–	0.074	–	0.027	–

†single seed, REAL + typo-injected slice only (GPU cost); other rows are 3-seed means. ‡real slice only, disclosed protocol asymmetries (§5.7): Baran uses oracle error positions + gold labels; Jellyfish is our detect-then-impute composition with seen-data caveats.

no-reference over-merging failure the grounding was built to prevent. The shipped verified-union system's suite row (REAL-F1 0.142, damage 0.015) shows the grounding wrapper and heuristic union carry entity canonicalization on these datasets — the model's contribution concentrates on the synthetic regime and hospital repair (§5.3), and the verifier cuts its suite damage to 0.015, 7× below the grounded heuristic's 0.104. Within our own ablations, removing grounding cedes 22% of real-error F1 (0.135 vs 0.174) and forfeits the behavioral guarantees: perfect abstention on adversarial traps (1.000) versus 0.250 without abstention, and reference-vetoed wrong merges (e.g. `guntxrsvillx`→`huntsville`).

5.3 Plan-level selective prediction on real errors

On hospital's 509 real errors, the verifier transforms the fine-tune from unshippable to precise (Figure 1): the raw model plan repairs 0.475 of errors at 0.185 precision; gated at τ=0.5 it reaches 0.993 precision at 0.287 coverage (146 of 147 committed changes correct). The union with the grounded heuristic buys coverage back: **0.905 precision at 0.413 coverage** (232 changes, 210 correct). This turns the system's promise into a measured sentence: *zero-configuration and zero labels, repair 41% of real errors at* $\geq$0.90 *precision, with every declined merge surfaced for review.* For context, Baran given oracle error positions and 20 gold-labeled tuples per dataset reaches 0.811 F1 on the same slice (§5.7)—selective prediction does not close a supervised gap, but it makes the zero-label operating point trustworthy, which is the regime our user occupies. Precision is flat (0.89–0.91) for $\tau \in [0.2, 0.8]$, so the operating point is not threshold-brittle, and the result is seed-robust: across three training seeds of the same data recipe the union operating point is 0.891 ± 0.012 precision at 0.396 ± 0.025 coverage (the shipped adapter is the strongest seed), with every seed clearing the 0.70-precision/0.30-coverage bar decisively.

Candidate-constrained planning (negative result). We also tested constraining the planner's *inputs*: the profiler emits evidence-backed (variant → canonical) candidate pairs (frequency dominance, edit similarity, reference membership) and the model may only select among them, with a deterministic check dropping off-candidate mappings to review flags. As a standalone guard it is strong—the raw plan's precision rises from 0.185 to 0.760 with no verifier at all—but composed with the verifier and union it reaches 0.876 precision at 0.387 coverage, slightly *below* the unconstrained pipeline at the same τ: the candidate cap (top-3 per surface) removes some correct repairs the verifier would have kept, and the two mechanisms gate the same failure class. We ship the

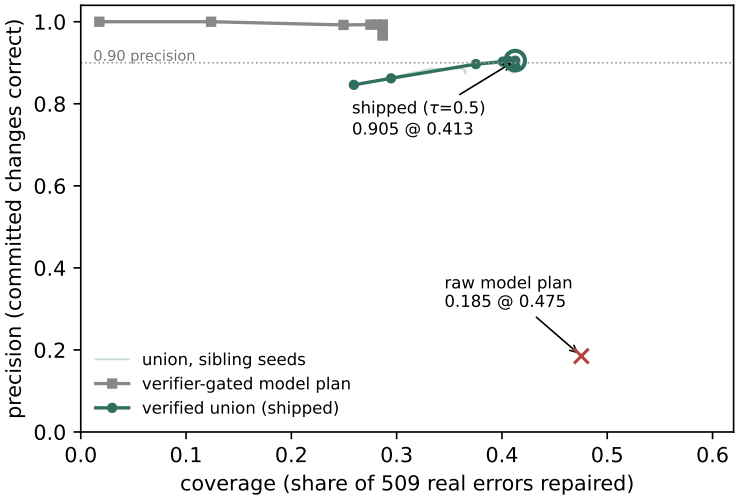

Figure 1: Plan-level precision–coverage on hospital (509 real errors), sweeping the verifier threshold τ. The union planner dominates the raw model plan; the shipped operating point (τ=0.5) is annotated.

verifier and keep candidate constraining available but off by default, reporting this as a measured redundancy rather than a stacked win.

5.4 Generalization to never-seen tables

The freeze-version system above was then pointed at data it had never seen, under three new harnesses (all released with this paper as the WildClean bundle). **(1) Paired bench**: 42 dirty/gold pairs spanning the Raha suite, SemTab ToughTables, government open-data typo corpora, entity-matching tables, and LLM-cleaning evaluation sets. On the 35 pairs from sources absent from training, the post-freeze system scores **macro F1 0.363 at damage 0.0219**. The largest single contribution is the regime §3.5(b) unlocks: on five all-unique entity tables where no in-column frequency signal exists, F1 moves from 0.0 to 0.955–0.957 at zero damage. Cross-row voting (§3.5c) is the second: flights—many sources reporting the same flight—goes from 0.044 to 0.164 F1 heuristic-only, and the heuristic hospital path doubles from 0.092 to 0.186. The hospital union gate is invariant under all of this (0.905 at 0.413). **(2) Wild bench**: 35 uncurated in-the-wild tables (open-data portals, GitHub, Kaggle) with no gold; we score seeded inject–recovery on each table's own data (mean recovery 0.207) plus a behavioral audit: every run yields a valid plan, every changed cell is attributable to a logged operation — **zero silent edits across all 35 tables**. **(3) Trust audit at scale**: 239 GitTables tables, same property — 239/239 valid plans, zero crashes, zero silent edits. The held-out-source generalization metric (train and evaluation drawn from disjoint benchmark sources) remains low in absolute terms (GEN-F1 0.058, variant-recall 0.108, damage 0.036): cleaning unfamiliar tables is far from solved, and we report the number to anchor the next section's claim about *where* the capability that does exist actually lives.

5.5 Weights do not carry generalization: negative training results

Every attempt to move never-seen-table performance through the model weights failed; every gain in §5.4 came from deterministic machinery plus the verifier. Five further supervised fine-tunes

— adding 109k harvested real-world alias pairs (ToughTables-derived, MusicBrainz search hints, RxNorm, OpenFlights), error-dense episode mixes, and a suspects-contract retrain — left held-out GEN-F1 flat or worse than the original recipe; mixing harvested pairs into the training blend *diluted* the synthetic skill the executor verifies (a monotonic dilution law across mix ratios). A GRPO pilot using the executor as a verifiable reward (the direction RLVR table work [14] motivates) was negative in all three arms at 4B/LoRA scale: the main arm and a KL-anchored variant degraded plan-format validity, and a random-reward control arm reproduced the same drift, identifying it as an RL artifact rather than signal [15]. We report this as the paper's central empirical claim rather than a failure of effort: in this protocol class, at this scale, **the planner's weights contribute approximately nothing to generalization beyond format competence** — profiling visibility, reference grounding, cross-row consensus, convention conservatism, and plan-level verification carry it. Concurrent work reaches the same conclusion from three independent directions [11, 12, 13]. The practical corollary is unusual but actionable: a contributor who wants to improve a system like this should write a deterministic capability and gate it with the verifier, not collect more training data.

5.6 Ablations

All ablations are 3-seed means (CIs $\leq \pm 0.003$). Removing abstention costs -0.013 NORTH, raises damage to 0.108 (from 0.104), and collapses trap abstention to 0.250. Removing the ambiguity margin costs -0.005 with $+0.001$ damage. Removing case matching costs -0.002 under the churn-neutral metric (and *gained* $+0.12$ under the uncorrected metric—the artifact). Replacing grounding with frequency clustering gains $+0.021$ NORTH, all of it from the injected slice (§4), while ceding -0.039 real-error F1—the trade the system refuses by design.

5.7 Learned-repair baselines under disclosed protocols

We additionally run two learned-repair baselines on the real-error (Raha) slice, under the identical churn-neutral metric but with honestly disclosed protocol asymmetries. **Baran** [1] is semi-supervised: we run its reference configuration—oracle error positions from the dirty/gold diff plus 20 gold-labeled tuples per dataset (its package default), without the optional Wikipedia-pretrained value models. It reaches REAL-F1 0.811 ± 0.018 (3 label-sampling seeds) at 0.003 damage—an upper bound under a strictly more informed protocol than ours (zero labels, no oracle detection); with oracle positions it can essentially only edit true-error cells, so its near-zero damage is structural. **Jellyfish-13B** [5] publishes per-cell error detection and imputation but no repair task; we compose the two (detect, then impute flagged cells with the attribute masked) — a pipeline of our construction, not theirs. It scores REAL-F1 0.074 at 0.027 damage (single seed, recommended decoding; note hospital is in its instruction-tuning data and flights/rayyan in its published evaluation suite, so these numbers may flatter it). Neither baseline is run on the 56-spec injected suite (computationally and methodologically out of scope for semi-supervised and per-cell-LLM repair); their NORTH/INJ-F1 cells in Table 1 are blank by design. The comparison locates our contribution: zero-config systems (ours, OpenRefine) occupy a different protocol class from supervised repair, and the verifier (§5.3) is what makes the zero-config class precise enough to trust, not what closes the labeled gap.

5.8 Calibration of abstention

On a probe of reference-entity typos plus garbage traps, retrieval confidence is a usable selective-prediction signal: AURC 0.120, ECE 0.169 (mildly over-confident), and precision rises monotoni-

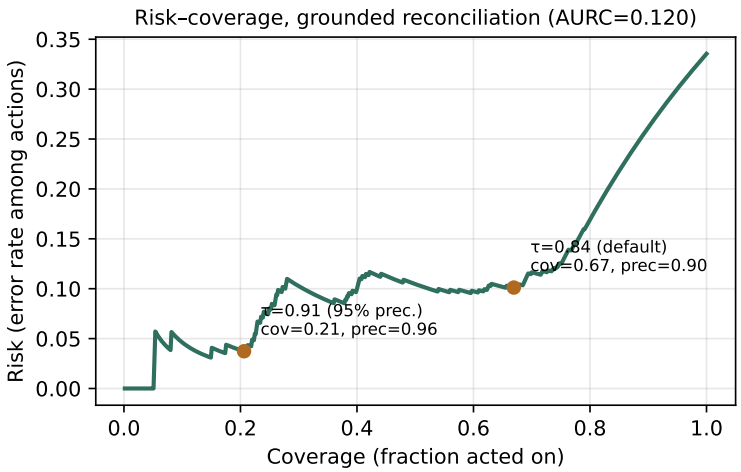

Figure 2: Risk–coverage for grounded city reconciliation (650 probes). Operating points annotated; the confidence supports thresholded abstention.

cally with threshold—0.899 precision at the default τ=0.84 (coverage 0.669), and $\geq$95% precision at τ=0.91 (coverage 0.206). Figure 2 shows the risk–coverage curve.

5.9 PII transfer and detection

The 44M OpenMed-PII classifier, trained on sentence-level clinical text, transfers to bare cell values without templates: 100% detection on person-name cells and 100% on addresses. Raw entity hits on non-PII columns (43%) are quasi-identifier typings (`city`, `occupation`) eliminated by the sensitive-type allowlist and coverage vote. The deterministic validator tier, evaluated out-of-distribution on per-type columns built from the Gretel PII test split (chosen specifically because OpenMed trains on Nemotron-PII), types **5/5** covered PII types correctly with **0/7** false positives on negative columns drawn from real open data. After deterministic masking, re-running all validators over the output finds zero (0/360) residual detectable PII.

6 Limitations

Reference coverage is the recall ceiling: entities absent from the taxonomy abstain by design, which is safe but not helpful; coverage work (larger gazetteers, ROR for organizations) moves recall directly. Our damage metric is convention-tolerant for case and whitespace but still counts alias expansion (`NYC`→`New York`) as damage when the gold keeps the alias—a value-level convention question we leave open. The confidence signal is mildly over-confident (ECE 0.169); temperature scaling is future work. The injected half of the suite, while seeded and reproducible, inherits the injector's error model; we mitigate with the real-error slice and report both. PII coverage is English-only, and we make no de-identification guarantee. Finally, the fine-tune headline is reported with multi-seed confidence intervals, but the wide-suite model row is single-seed for cost reasons and scoped as such.

7 Conclusion

A small, locally-run planner with execution-verified supervision, reference grounding, and calibrated abstention turns LLM data cleaning from a trust liability into an auditable system: every change is a named, reversible operation; uncertain actions become review flags rather than silent corruptions; and the evaluation itself is built to resist gaming. The post-freeze program sharpened the recipe into a finding: across five further fine-tunes and a three-arm GRPO pilot, the weights never moved never-seen-table performance — deterministic visibility, grounding, consensus, and verification did, at zero silent edits across 35 wild tables and a 239-table trust audit. We believe the recipe—propose/execute decomposition, verification-by-execution, retrieval-grounded outputs, and selective prediction over deterministic capabilities—is a template for deploying small specialized models on other structured tasks.

Reproducibility

Code, evaluation suite, result artifacts, and the fine-tuned weights are public (repository linked from the demo Space). Each reported number regenerates from one command at the released revision: the money table (`python -m eval.run_real_multi --out ...`), the precision–coverage curve and shipped operating point (`python -m eval.precision_curve --plan eval/results/v6_hospital_raw_plan.js --union`), Baran (`python eval/run_baran.py` in the pinned environment documented in that file, then `python -m eval.baselines_learned --score-baran`), Jellyfish (`modal run scripts/modal_jellyfish.p` ablations (`python -m eval.ablations`), calibration (`python -m eval.calibration`), and the PII leak test (`python -m eval.pii_leak`). The generalization harnesses regenerate likewise: paired bench (`python -m eval.paired_bench`), wild bench (`python -m eval.wild_bench`), GitTables trust audit (`python -m eval.gittables_audit`), and held-out-source generalization (`python -m eval.generalization`). The WildClean bundle — redistributable dirty/gold pairs, the GitTables audit slice, open vocabularies, result JSONs, and license-gated loaders for the non-redistributable pairs — is released as a Hugging Face dataset. The shipped product planner is the identical code path measured here (`scrubdata/active.py`).

References

[1] M. Mahdavi et al. Raha: A configuration-free error detection system. SIGMOD 2019; Baran: Effective error correction via a unified context representation. VLDB 2020.

[2] T. Rekatsinas, X. Chu, I. Ilyas, C. Ré. HoloClean: Holistic data repairs with probabilistic inference. VLDB 2017. arXiv:1702.00820.

[3] J. Peng et al. GARF: Self-supervised interpretable data cleaning. VLDB 2023.

[4] A. Narayan, I. Chami, L. Orr, C. Ré. Can foundation models wrangle your data? VLDB 2022. arXiv:2205.09911.

[5] H. Zhang et al. Jellyfish: A large language model for data preprocessing. EMNLP 2024.

[6] Anonymous. Cocoon: Semantic data cleaning with LLMs. arXiv:2410.15547, 2024 (unreviewed preprint).

[7] ZeroED: Hybrid zero-shot error detection through LLM reasoning. arXiv:2504.05345, ICDE 2025.

[8] Ensembling LLM-induced decision trees for explainable and robust error detection. arXiv:2512.07246, 2025.

[9] Q. Chen et al. Auto-Test: Learning semantic-domain constraints for unsupervised error detection. SIGMOD 2025. arXiv:2504.10762.

[10] GIDCL: Generative-discriminative iterative data cleaning with LLMs. SIGMOD 2025.

[11] Spreadsheet-RL: Reinforcement learning for spreadsheet manipulation with execution verification, 2026.

[12] An empirical study of LLM robustness to tabular distortions. arXiv:2601.05009, 2026.

[13] When helping hurts and how to fix it: Multi-agent debate for data cleaning. arXiv:2606.02866, 2026.

[14] Table-R1: Region-based reinforcement learning with verifiable rewards for table reasoning. arXiv:2505.23621, 2025.

[15] Spurious rewards: Rethinking training signals in RLVR. arXiv:2506.10947, 2025.

[16] P. Li et al. Table-GPT: Table-tuned GPT for diverse table tasks. SIGMOD 2024.

[17] M. Ahmad et al. RetClean: Retrieval-based data cleaning using LLMs and data lakes. VLDB 2024. arXiv:2303.16909.

[18] X. Deng et al. TURL: Table understanding through representation learning. VLDB 2020. arXiv:2006.14806.

[19] T. Zhang et al. TableLlama: Towards open large generalist models for tables. NAACL 2024. arXiv:2311.09206.

[20] F. Belotti et al. Evaluating LLMs on entity disambiguation in tables. 2024. arXiv:2408.06423.

[21] L. Qiu et al. RACOON: Retrieval-augmented column type annotation with a knowledge graph. 2024. arXiv:2409.14556.

[22] P. Nguyen et al. MTab: Matching tabular data to knowledge graph using probability models. SemTab 2019. arXiv:1910.00246.

[23] R. El-Yaniv, Y. Wiener. On the foundations of noise-free selective classification. JMLR 2010; Y. Geifman, R. El-Yaniv. Selective classification for deep neural networks. NeurIPS 2017.

[24] M. Panahi. OpenMed NER: Open-source domain-adapted token classifiers for biomedical NER. 2025. arXiv:2508.01630.